E-Commerce Order Analytics System

Celebal Summer Internship 2026 – Week 8

Submitted by: Aditya Kumar, 1000019772@dit.edu.in

Introduction

The Week 8 assignment focuses on building an E-Commerce Order Analytics System using Python and SQL in a local environment. The project simulates a realistic business scenario in which raw online-order data contains quality issues and must be cleaned, validated, transformed, stored in a relational database, and analyzed to generate useful business insights.

The implementation covers the complete workflow: synthetic data generation, data cleaning and validation, SQLite database loading, basic and advanced SQL analytics, command-line reporting, and edge-case testing.

Assignment Requirements

The assignment requires four relational CSV datasets, intentional data-quality issues, Python-based cleaning, SQL analysis, a CLI reporting tool, and four edge-case tests.

Requirement	Implementation
Four CSV datasets	customers.csv, products.csv, orders.csv, order_items.csv
Minimum dataset size	All four datasets contain at least 500 records
Intentional data issues	Missing customer IDs, negative quantities, malformed dates, product-name defects, invalid emails, orphan items
Data cleaning	Implemented with Python and Pandas
Database	SQLite with relational constraints and integrity checks
SQL analysis	17 analytical SQL queries
CLI reporting	Daily, weekly, and monthly reporting with date-range support
Edge cases	Orphan items, excessive discounts, zero quantity, future dates

Project Architecture

The project is organized as a local Python + SQL pipeline:

Raw CSV Data
↓
Data Generation
↓
Cleaning & Validation
↓
Cleaned CSV Data
↓
SQLite Database
↓
SQL Analytics
↓
CLI Business Reports
↓
Edge-Case Testing

The implementation intentionally remains local and lightweight. No web application, API layer, cloud infrastructure, or external service is required.

- **Data Generation**

A deterministic Python generator creates four related CSV datasets with realistic synthetic e-commerce data and intentional anomalies.

Dataset	Rows	Purpose
customers.csv	1,000	Customer identity, contact, registration, and customer type
products.csv	500	Product catalog, categories, subcategories, and pricing
orders.csv	1,000	Order dates, customer references, status, and region
order_items.csv	2,000	Products purchased, quantities, prices, and discounts

- **Intentional Data Issues**

Issue	Generated Result
Missing customer_id	50 orders (5%)
Negative quantities	60 order items (3%), representing returns
Malformed order dates	50 orders using DD-MM-YYYY format
Product-name defects	30 product names with whitespace/casing issues
Invalid emails	20 customer records (2%)
Orphan order items	30 items referencing non-existent orders

The generator uses a fixed random seed of 42, making the raw dataset reproducible.

- **Data Cleaning and Validation**

The cleaning pipeline processes the raw CSV files and produces cleaned datasets together with a data-quality report.

Function	Implementation
clean_orders()	Corrects malformed dates and preserves valid guest checkouts with missing customer_id
clean_products()	Trims whitespace and normalizes product names
validate_emails()	Identifies invalid customer email addresses
check_referential_integrity()	Detects order_items referencing non-existent orders

- **Cleaning Results**

Output	Result
Cleaned customers	1,000
Cleaned products	500
Cleaned orders	1,000
Valid cleaned order items	1,970
Orphan items isolated	30
Malformed dates corrected	50
Invalid emails detected	20
Negative return quantities preserved	60

Orphan order items are not silently discarded; they are isolated in orphan_order_items.csv. Guest checkout orders with NULL customer_id are preserved because they represent valid business records.
A data-quality report is generated in data/cleaned/data_quality_report.md containing the detected issues and cleaning results.

- **SQLite Database**

The cleaned datasets are loaded into ecommerce.db using SQLite. The database uses relational keys and foreign-key enforcement to maintain data integrity.

Table	Rows
customers	1,000
products	500
orders	1,000
order_items	1,970

Database validation results:

- PRAGMA integrity_check: ok
- PRAGMA foreign_key_check: 0 violations
- All valid order_items reference existing orders and products
- Guest orders remain valid with NULL customer_id

- **SQL Analysis**

The project implements all 17 analytical SQL requirements from the assignment.

No.	Query	Technique / Purpose
1	Revenue per Category	Aggregation
2	Top 10 Customers	Aggregation and ordering
3	Last 12 Months	Date filtering and grouping
4	Undelivered Customers	Conditional aggregation
5	Excessive Returns	Aggregation and HAVING
6	Category Return Rate	Conditional aggregation
7	Running Totals	Window SUM()
8	Dense Rank	DENSE_RANK()
9	LAG / LEAD Analysis	LAG() and order-gap analysis
10	Multi-Level CTE	CTE-based customer segmentation
11	NTILE Segmentation	NTILE(4) lifetime-value quartiles
12	Year-over-Year Comparison	Calendar-year comparison
13	First / Last Value	FIRST_VALUE() / LAST_VALUE()
14	Cumulative Distribution	Cumulative revenue contribution
15	Cohort Analysis	Registration-month retention
16	Self-Join + Window	Consecutive-order analysis
17	Frequently Bought Together	Product-pair analysis

All 17 SQL queries were executed successfully against the final SQLite database.

- **Advanced SQL Implementation**

  - **Running Totals**
    Daily revenue is aggregated by region and a cumulative running total is calculated in date order using a window function -> *SUM(daily_revenue) OVER (PARTITION BY region_code ORDER BY order_date)*

  - **Dense Ranking**
    Products are ranked by total revenue within each category. Products with equal revenue receive the same rank using DENSE_RANK().

  - **LAG Analysis**
    Consecutive customer orders are compared using LAG() to calculate days between orders. Customers with an average gap greater than 30 days are flagged as At Risk.

  - **CTE Segmentation**
    Monthly customer revenue is first calculated and then categorized into High, Medium, and Low segments according to the assignment thresholds.

  - **NTILE Segmentation**
    Customers with purchasing activity are divided into four lifetime-value quartiles using NTILE(4), labelled Platinum, Gold, Silver, and Bronze.

  - **Cohort Analysis**
    Customers are grouped by registration month and order activity is tracked across month 0 through month 3 to calculate retention.

  - **Frequently Bought Together**
    Product pairs appearing in the same order are identified using a self-join. Same-product pairs and reversed duplicates are excluded by enforcing an ordered product-pair condition.

- **Python + SQL Integration**

  A command-line reporting tool connects directly to the SQLite database and generates business summaries for a selected period.

Feature	Implementation
Report types	Daily, Weekly, Monthly
Date range	Start and end dates
Database	SQLite
Metrics	Total orders, revenue, unique customers
Products	Top 3 products
Comparison	Previous-period percentage change
Dependencies	Python standard library and sqlite3

Example execution: *python src/cli/report.py --type Monthly --start 2023-01-01 --end 2023-01-31*
The CLI was also tested with an empty date range and handled the absence of sales without crashing.

- **Edge Case Handling**

Test	Result
Orphan order_item	Detected and isolated without failing
discount_percent > 100	Handled safely and verified
quantity = 0	Processed safely with zero revenue contribution
Future order_date	Detected successfully

All four edge-case tests passed using Python unittest.

- **Final Verification**

Component	Status
Synthetic data generation	PASS
Data cleaning and validation	PASS
Raw data preservation	PASS
SQLite database loading	PASS
SQLite integrity check	PASS
Foreign-key check	PASS — 0 violations
SQL analytics	PASS — 17/17
CLI reporting	PASS
Edge-case tests	PASS — 4/4

- **Conclusion**

The Week 8 E-Commerce Order Analytics System successfully implements the complete local Python and SQL workflow specified in the assignment. The project generates relational synthetic data with intentional defects, cleans and validates the data, enforces database integrity, performs the required analytical SQL operations, integrates SQL with a command-line reporting tool, and verifies the specified edge cases.
The final implementation demonstrates practical skills in Python data processing, relational data modeling, SQL analytics, window functions, CTEs, customer segmentation, cohort analysis, command-line integration, and defensive testing.